

PRINCIPLES OF PROGRAMMING LANGUAGES / LESSON GUIDE

02 Iteration

Modules 1-2: control flow, state, and traversal

1. Introduction & core concept

Iteration is repeated evaluation governed by a continuation rule and a progress mechanism. A loop transforms a program state: the current position, accumulator, and relevant objects change as the body executes. Languages provide iteration because repetition over an input-dependent number of items cannot be expressed conveniently by duplicating statements.

Count-controlled loops expose an index and bound. Condition-controlled loops continue while a predicate holds. Collection traversal requests successive elements through an iteration mechanism. A pre-test loop may execute zero times; a post-test loop evaluates its condition after the body. The categories describe control, not the computational cost of the body.

2. Under the hood: the mechanics

Parsing records the condition or iterable, body, and control transfers. Execution can be compiled into tests and jumps or handled through bytecode; the language does not prescribe one physical instruction sequence. Python for obtains an iterator and assigns successive values to its target [1]. JavaScript counter loops explicitly sequence initialization, test, body, and update [2].

A loop invariant describes what remains true at a chosen point in every iteration. A termination measure explains progress, such as the number of unprocessed items. These obligations differ: a loop can preserve an invariant forever. A loop does not inherently allocate another function frame on each repetition, but it can retain growing collections or closures.

Learning outcomes

- Trace changes in loop state and identify a valid invariant.
- Separate traversal count from the cost of work inside the body.
- Explain safe progress when a collection changes during traversal.

Examples progress from Python 3 to modern JavaScript to C++17. Code follows each language's native syntax. Bracketed citations refer to the final page.

3. CODE EXAMPLES / PYTHON

Simple: accumulate a finite sequence

```
def sum_to(n):
    if type(n) is not int or n < 0:
        raise ValueError("Use a nonnegative integer")
    total = 0
    for number in range(1, n + 1):
        total += number
    return total

print(sum_to(5))
print(sum_to(0))
```

In-depth reasoning

Expected result: 15
0

Calling `sum_to` creates bindings for `n` and `total`. The range describes integers from 1 through `n`; it is not a list prefilled with every value. The loop assigns each yielded integer to `number` and updates `total`. For input 5, the accumulator changes from 0 to 1, 3, 6, 10, and 15.

The invariant is: `total` equals the sum of all values already consumed. Initially the processed prefix is empty, so zero is correct. Adding the next value preserves the invariant. When the finite range is exhausted, the entire required prefix has been processed. For zero, the range is empty and the body never executes.

There are `n` additions and a bounded number of scalar bindings. Under unit-cost arithmetic this is $O(n)$ time and $O(1)$ auxiliary scalar state. Python integers can grow, so exact bit costs depend on operand size. Validation separates the supported domain from accidental behavior for negative or non-integer inputs.

Compare the language models

Python's loop consumes an iterable. JavaScript and C++ can express the same recurrence with explicit counter loops, but their ordinary numeric types and overflow or precision behavior differ. Equivalent output for a small example does not imply identical representations.

Checkpoint: What changes if the upper range bound is `n` instead of `n + 1`? The required final term is omitted.

3. CODE EXAMPLES / JAVASCRIPT

Intermediate: preserve per-iteration checkpoints

```
function checkpoints(values) {
  let total = 0;
  const reports = [];
  for (let i = 0; i < values.length; i++) {
    if (values[i] < 0) {
      continue;
    }
    total += values[i];
    const snapshot = total;
    reports.push(() => `${i}: ${snapshot}`);
  }
  return reports;
}

for (const report of checkpoints([4, -2, 7, 0])) {
  console.log(report());
}
```

In-depth reasoning

Expected result: 0: 4

2: 11

3: 11

Assume a fixed ordinary array of finite numbers. The loop reads indices in order. A negative value triggers `continue`, which skips the rest of the body but still permits the counter update. Accepted values contribute to the shared total. Zero is accepted because it is not negative.

Each saved function is called after `checkpoints` returns. The `let` loop variable has per-iteration binding behavior; `snapshot` is initialized separately for each accepted body execution. The functions therefore retain the intended index and accumulated value. Capturing `total` directly would make them all read the final shared total instead.

Changing `let i` to `var i` would give these closures a shared function-scoped counter, whose value is 4 when they run. The algorithm scans n inputs and retains k reports: $O(n)$ construction time under ordinary array assumptions and $O(k)$ retained state. Iterative syntax does not make storage constant.

Compare the language models

An ordinary Python `for` loop does not introduce a separate lexical scope on every iteration, so analogous closures require deliberate value capture. C++ lambdas make value versus reference capture explicit; retained references must not outlive their referents.

Checkpoint: Why does `continue` not cause an infinite loop here? The `for` statement still executes `i++`.

3. CODE EXAMPLES / C++

Advanced: erase without losing traversal state

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> values{-3, 4, -1, 7, 0, -5, 2};
    auto it = values.begin();
    while (it != values.end()) {
        if (*it < 0) {
            it = values.erase(it);
        } else {
            ++it;
        }
    }
    for (int value : values) {
        std::cout << value << ' ';
    }
    std::cout << '\n';
}
```

In-depth reasoning

Expected result: 4 7 0 2 (with a trailing space)

The vector stores integers contiguously. `begin` identifies the first element, while `end` is the past-the-end position. The condition prevents dereferencing `end`. If the current element is negative, `erase` removes it and returns a usable iterator to the following element or the new end [3].

Erasure invalidates vector iterators and references at or after the removed position. Assigning the returned iterator is essential. The erased branch does not increment again: an unprocessed element has shifted into the current position. The kept branch advances normally. The invariant is that all elements before it have been processed and retained values are nonnegative.

Every repetition reduces the number of unprocessed original elements. However, erasing near the front shifts later elements. Repeated shifts can make total work $O(n^2)$, even though each element is examined only once. An empty vector safely skips the loop. A linear compaction strategy is preferable for large inputs.

Compare the language models

Python list removal and JavaScript array splicing can also shift elements and cause skipped values. C++ adds explicit iterator invalidation obligations: using an invalidated iterator is not a reliable exception-producing operation.

Checkpoint: Why not increment `it` after `erase`? Doing so can skip the next value, including a consecutive negative.

4. CONSOLIDATION / REFERENCES

Reason, compare, and apply

The central lesson

Correct iteration combines an invariant, a progress argument, and a cost model. A single loop may be quadratic; a loop may retain linear storage. Ask what is visited, what changes, and what each operation costs. In Paradigm Diagnostics, use the Iteration card to inspect an accumulator in all three languages, then compare it with these richer examples.

Review questions with answer guidance

1. Design an even-number sum and state its invariant.

Before each new item, the accumulator equals the sum of accepted even items in the processed prefix.

2. How can vector filtering become linear?

Move retained elements to successive write positions, then erase the unused suffix once.

A short learning activity

Predict each example's outcome before running it. Change one input or one operation, state the rule you expect to apply, and compare the observation with the prediction. Explain the earliest point where behavior changes. A matching output supports that particular execution; it does not prove general program equivalence.

References and further reading

[1] Python: for statement. [Read source](#)

[2] ECMAScript: iteration statements. [Read source](#)

[3] C++ working draft: vector modifiers. [Read source](#)

References are official language/runtime documentation or the public C++ working draft. Links checked 30 September 2026. Examples and reasoning are original teaching material; implementation details and scheduling outcomes are identified where relevant.